

1. Indicate whether the following statements are true or false. Justify your answers.

a) A ready process waiting to get access to the CPU is in the “waiting” state.

b) When any hardware interrupt (not a trap) arrives, the process that is currently in the “running” state (if any), will be moved to the “ready” state.

c) The “wait()” system call is generally used by a child process to wait for instructions from a parent process.

d) Message passing based Inter-Process Communication (IPC) consumes less memory than shared memory based IPC.

1a) A ready process waiting to get access to the CPU is in the “waiting” state.

→ False.

Justification: The process is in the “ready” state. A process in the “waiting state” is waiting for an event or the completion of an I/O operation.

1b) When any hardware interrupt (not a trap) arrives, the process that is currently in the “running” state (if any), will be moved to the “ready” state.

→ True.

Justification: The running process will be moved to the "ready" state so that the Interrupt Service Routine (ISR) can be executed. After completion of the ISR, either the same process or another process from the ready queue can be selected for execution by the OS.

Note: In some OS implementations, the state of the running process in the PCB may not be updated to "ready" when executing the ISR. This is usually in situations when the OS knows that the same process will continue its execution after completion of the ISR.

1c) The “wait()” system call is generally used by a child process to wait for instructions from a parent process.

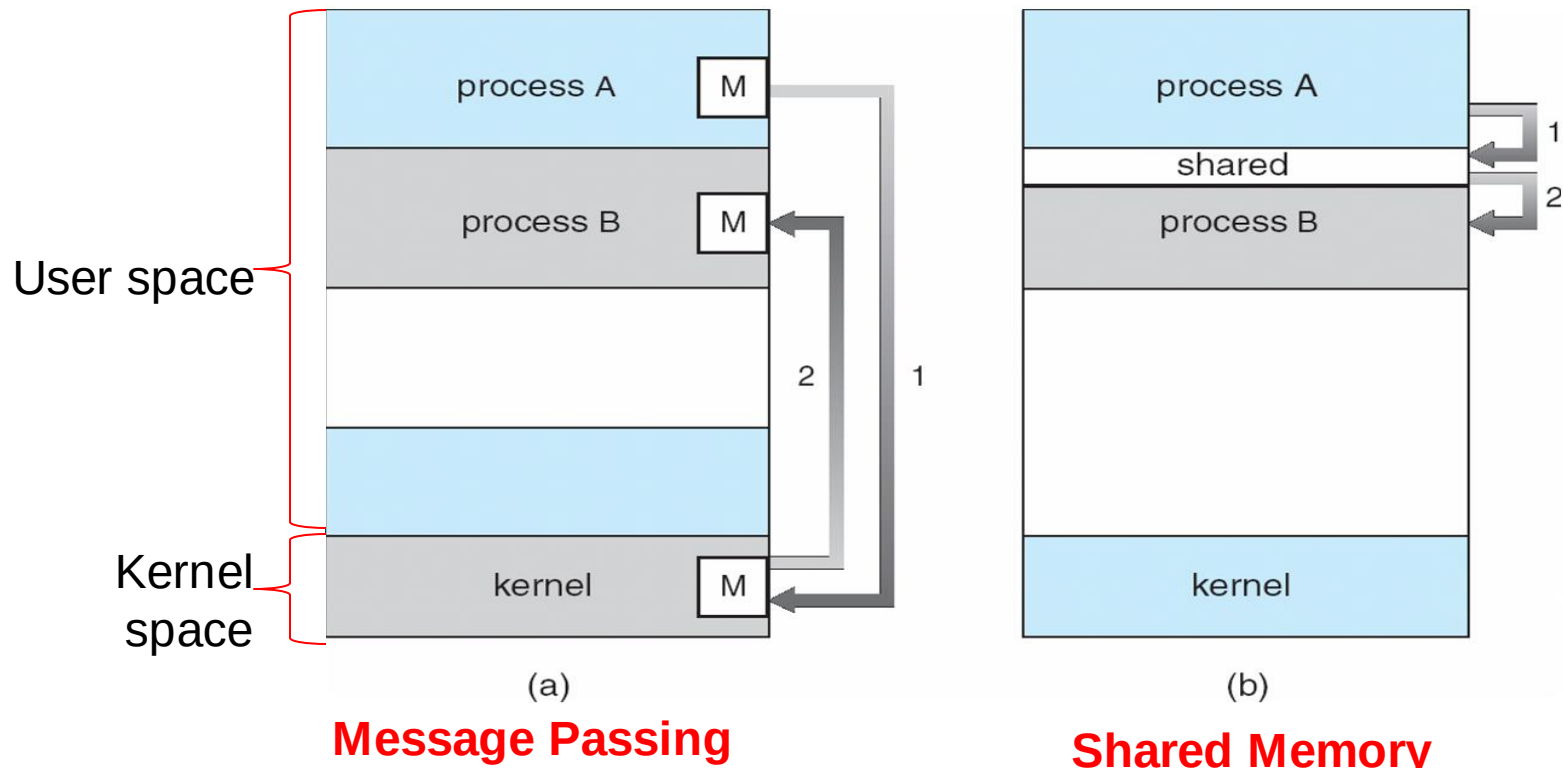
→ False.

Justification: It is used by a parent process to wait for the completion of a child process. Sometimes it is also called “join()”, a reference to the popular fork-join processing model. In this model, a parent process forks several children processes for independent computations, and then combines their results using the join() operation.

1d) Message passing based Inter-Process Communication (IPC) consumes less memory than shared memory based IPC.

→ False.

Justification: It consumes more memory.



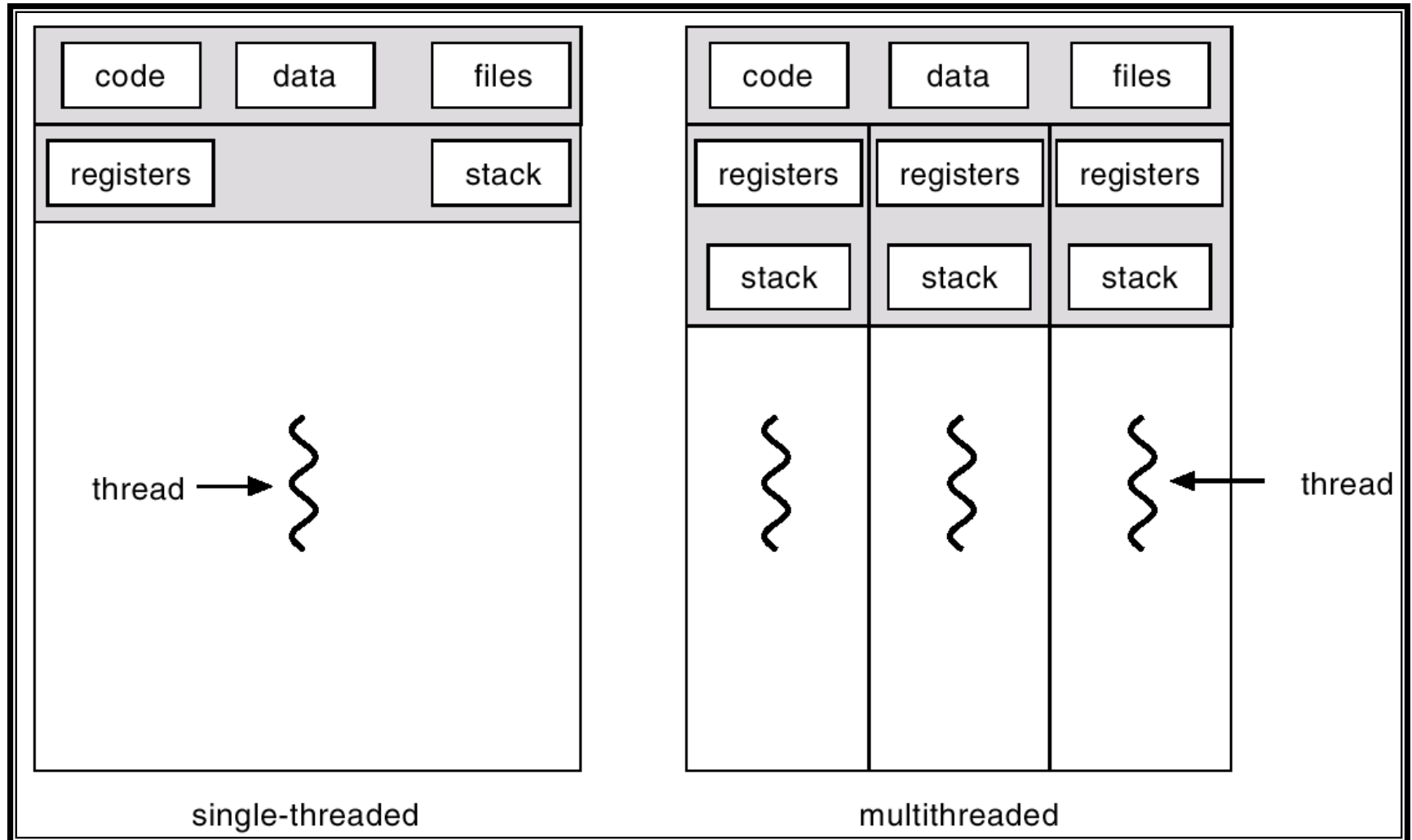
2. What are two main differences between the data and stack regions of a process memory?

Data vs. Stack Regions of Memory

1. The size of the data region of a process memory is statically fixed, whereas a stack can grow and shrink as the process executes.
2. The data region is used for storing global parameters/variables, whereas the stack region is used for local parameters/variables in functions.

3. Explain the difference between a single-threaded and a multi-threaded process.

Single- vs. Multi-threaded Process

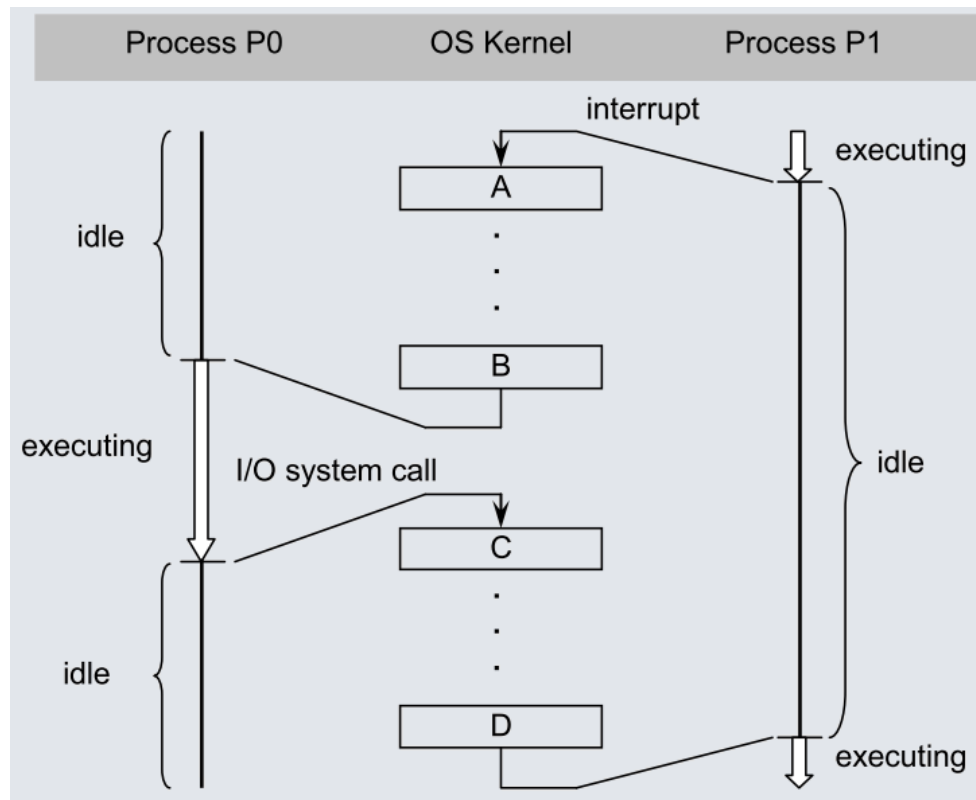


Single- vs. Multi-threaded Process

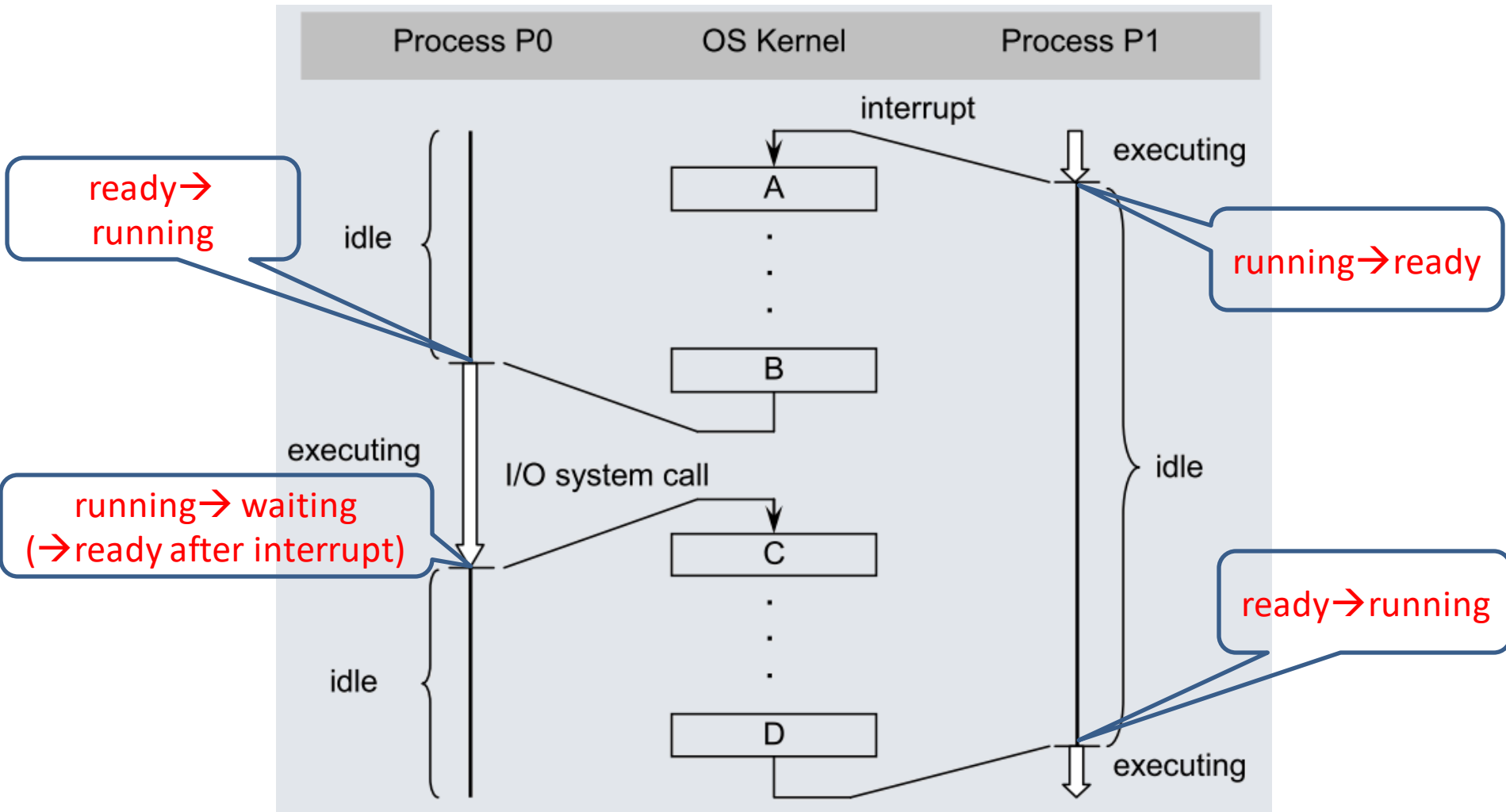
- Threads in a process share code, data and heap regions of memory, whereas stack space is unique to each thread. Also, each thread has its own Thread Control Block (TCB), similar to a PCB.
- In a single-threaded process, there is only one thread of execution, and hence it is identical to a process.
- In a multi-threaded process, the individual threads can execute concurrently, thus increasing system throughput; when one thread of a process is blocked (“waiting” state), another thread can continue its execution (“running” state).

4. The figure below shows the execution of processes P0 and P1 in a multiprogramming system.

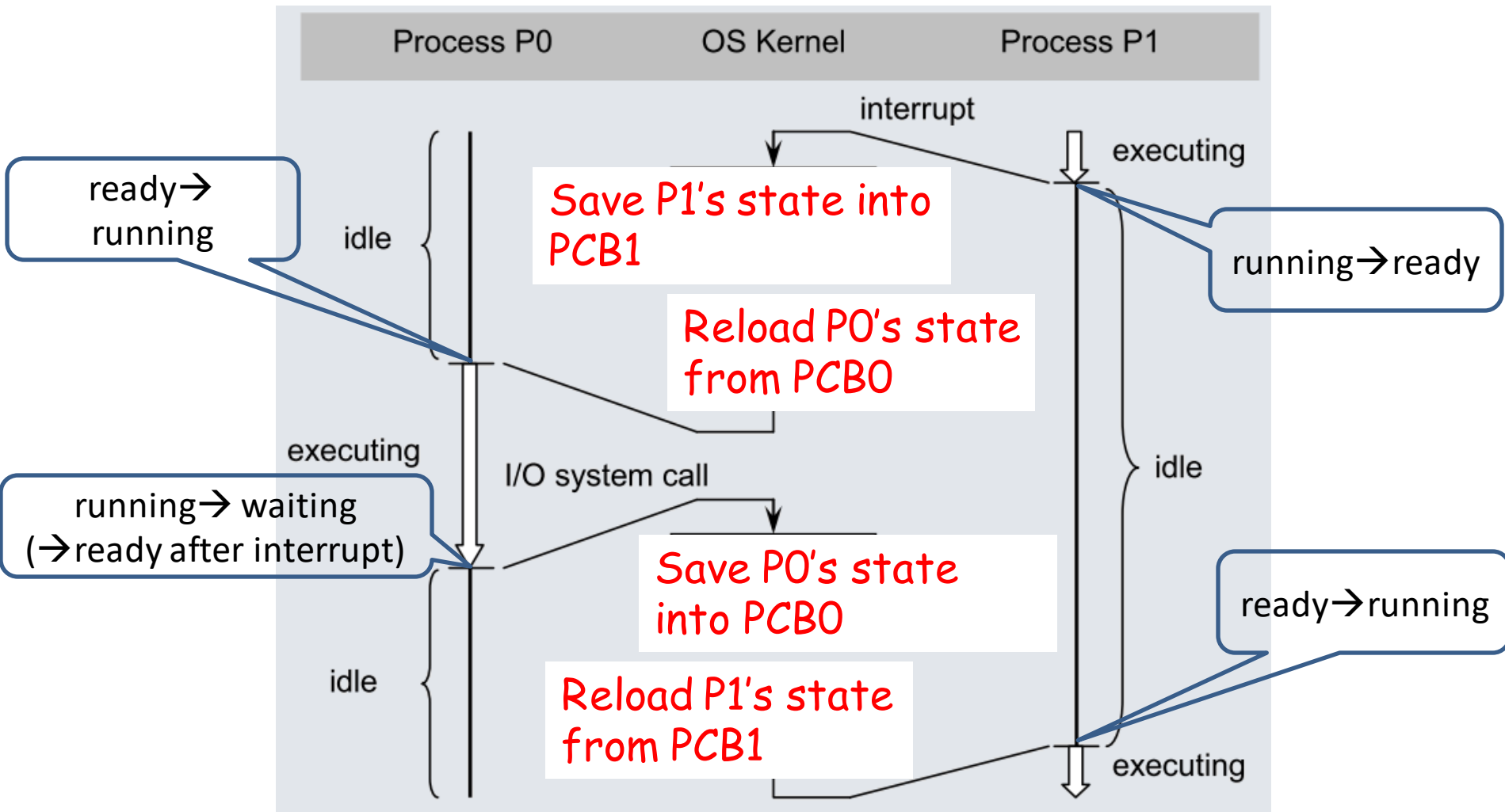
- Identify state transitions of each process.
- Describe operations A, B, C and D performed by the operating system kernel.



State Transitions



Operations A, B, C and D



State Transition Options

- Process P0
 - The first transition for P0 is from "ready" to "running"
 - It is also possible that in fact this is a transition from "waiting" to "ready" to "running" (the "waiting" to "ready" transition occurring in response to the interrupt that arrived – example, the interrupt signals end of an I/O operation previously initiated by P0)
- Process P1
 - Is it possible that the first transition for P1 is from "running" to "waiting" and then to "ready"?
 - This can happen if the interrupt mentioned in the diagram is actually an I/O system call (it is a trap which is mentioned as an interrupt). However, if this was the case, there must be some event that would allow P1 to transition from "waiting" to "ready" so that it can eventually go to "running". There is no such event in the diagram, and hence the first transition for P1 has to be from "running" to "ready"

Exact sequence of steps in A--B

- Interrupt arrives
- CPU pauses the "running" process, switches h/w to kernel mode and gives control to OS
- OS saves the context of the "running" process
- OS changes state of the process from "running" to "ready" in the process PCB
- OS identifies and executes ISR
- After completion of ISR, OS identifies next process to run from the ready queue
- OS changes state of this new process from "ready" to "running" in the process PCB
- OS switches h/w to user mode
- OS loads context of the new process (process in the "running" state)
- New process starts running on the CPU